UNIVERSITÀ DELLA CALABRIA

Algorithmic Game Theory — a.a. 2026/2027 — Sebastiano A. Piccolo

Appunti in italiano, parte di `main.pdf` — 3 ottobre 2026

LEZIONE 1

Introduzione all'Algorithmic Game Theory

TEORIA

1.1 Informazioni sul corso

- **Lezioni**: mercoledì 10:30–13:30 (teoria), giovedì 11:30–13:30 (laboratorio).
- **Teams**: *Algorithmic Game Theory 2026/27* (il codice per entrare è nelle slide).
- **Esame**: un **progetto** in cui si deve *formalizzare* un problema e *implementarne* una soluzione usando gli strumenti dell'AGT.

> **Osservazione (conseguenza pratica).** Dato che l'esame è un progetto, contano due cose: saper *modellare* un problema (chi sono i giocatori, quali azioni hanno, quali utilità) e saper *scrivere codice* che calcola il concetto di soluzione giusto. I laboratori vanno quindi studiati quanto la teoria.

Programma (provvisorio)

1. Giochi combinatori (*Combinatorial Games*)
2. Teoria dell'utilità (*Utility theory*)
3. Teoria dei giochi non cooperativi (*Non-cooperative game theory*)
4. Teoria dei giochi coalizionali (*Coalitional game theory*)
5. Scelta sociale computazionale (*Computational Social Choice*)
6. Mercati di abbinamento (*Matching Markets*)
7. Progettazione di meccanismi (*Mechanism Design*)
8. Divisione equa (*Fair division*)

Testi di riferimento

Non esiste un unico libro che copra tutto il corso; queste sono le risorse gratuite indicate:

- Y. Shoham, K. Leyton-Brown, *Multiagent Systems*;
- N. Nisan, T. Roughgarden, É. Tardos, V. Vazirani, *Algorithmic Game Theory*;
- A. Karlin, Y. Peres, *Game Theory, Alive*;
- F. Brandt et al., *Handbook of Computational Social Choice*.

1.2 Che cos'è la teoria dei giochi

Definizione 1.1 (Teoria dei giochi). La **teoria dei giochi** studia le *decisioni prese da più soggetti* (*multiperson decision making*), con lo scopo di prevedere come più agenti **razionali** interagiranno e si comporteranno in situazioni di **conflitto** e/o di **cooperazione**.

I quattro ingredienti della definizione:

1. **Conflitto**: le azioni di un giocatore influenzano gli altri.
2. **Cooperazione**: i giocatori possono unire le forze per raggiungere un obiettivo comune.
3. **Comportamento razionale**: ogni giocatore vuole *massimizzare* la propria **utilità attesa** (*expected utility*).
4. **Predizione**: vogliamo sapere cosa succederà. Lo si fa tramite i **concetti di soluzione** (*solution concepts*), come l'equilibrio di Nash o il valore di Shapley.

1.3 Che cos'è l'Algorithmic Game Theory

Definizione 1.2 (Algorithmic Game Theory). L'**Algorithmic Game Theory** (AGT) è l'area all'intersezione tra *teoria dei giochi* e *progettazione di algoritmi*, il cui obiettivo è progettare algoritmi in *ambienti strategici*. [Nisan et al. 2007]

In concreto, l'AGT si occupa di progettare algoritmi per:

- **calcolare** in modo efficiente i concetti di soluzione (es. trovare un equilibrio);
- **aggregare** voti e preferenze;
- **allocare** risorse scarse.

1.4 Panoramica delle aree dell'AGT

1.4.1 Giochi non cooperativi

I giochi non cooperativi modellano situazioni di **conflitto**: i giocatori competono tra loro e non possono necessariamente cooperare. Esempi: scacchi, sasso-carta-forbici, dilemma del prigioniero.

L'obiettivo è modellare una situazione e capire cosa succede. Il concetto di soluzione tipico è l'**equilibrio**.

Il dilemma del prigioniero

Due giocatori scelgono simultaneamente se *cooperare* (C) o *tradire* (D, *defect*). I numeri sono le utilità (anni di carcere con segno negativo): in ogni cella il primo valore è di P1, il secondo di P2.

		P2	
		Cooperate	Defect
P1	Cooperate	$-1, -1$	$-5, 0$
	Defect	$0, -5$	$-3, -3$

Tabella 1.1: Dilemma del prigioniero in *forma normale* (gioco bi-matrice).

Questa rappresentazione a tabella si chiama **forma normale** (o gioco *bi-matrice*): una matrice di utilità per ciascuno dei due giocatori.

Definizione 1.3 (Ottimo di Pareto). Un esito è **Pareto-ottimo** se non esiste un altro esito in cui *qualcuno* sta meglio e *nessuno* sta peggio. In altre parole: non si può migliorare la situazione di un giocatore senza peggiorare quella di qualcun altro.

Definizione 1.4 (Equilibrio di Nash, idea). Un profilo di strategie è un **equilibrio di Nash** se nessun giocatore può migliorare la propria utilità cambiando *da solo* la propria scelta.

Esempio (analisi del dilemma). Nash. Per P1, tradire conviene *qualunque* cosa faccia P2: se P2 coopera, $0 > -1$; se P2 tradisce, $-3 > -5$. Lo stesso vale per P2. Quindi (D, D) è l'unico equilibrio di Nash, con utilità $(-3, -3)$. Da (C, C), invece, ciascuno ha interesse a deviare (passa da -1 a 0). **Pareto.** (C, C) con $(-1, -1)$ è Pareto-ottimo. L'equilibrio (D, D) *non* lo è: (C, C) fa stare meglio entrambi.

Il dilemma mostra la tensione centrale della teoria dei giochi:

Ottimalità di Pareto ⇒ efficienza **Equilibrio di Nash ⇒ stabilità**

L'esito efficiente non è stabile e l'esito stabile non è efficiente.

Attenzione (definizione nelle slide). Le slide definiscono l'ottimo di Pareto come *"nessun giocatore può deviare senza peggiorare qualcun altro"*. È una formulazione informale: l'ottimo di Pareto *non* riguarda le deviazioni unilaterali (quelle sono il criterio di Nash), ma il confronto tra *esiti*. Nel dilemma sono Pareto-ottimi anche (C, D) e (D, C): passando da $(-5, 0)$ a qualunque altro esito P2 starebbe peggio. L'unico esito *non* Pareto-ottimo è (D, D).

Applicazioni

- algoritmi per giocare: scacchi, poker, . . . ;

- *Generative Adversarial Networks* (GAN): generatore e discriminatore sono due giocatori;
- sicurezza: pattugliamento, sorveglianza, . . . ;
- decisioni sequenziali robuste (*robust sequential decision making*).

1.4.2 Giochi coalizionali

Nei giochi coalizionali (o cooperativi) i giocatori **uniscono le forze** per un obiettivo comune, ad esempio mettono insieme i soldi per comprare un bene. Le domande sono due:

- quale coalizione è più probabile che si formi?
- come si divide il bene (il *payoff*) tra i membri?

Concetto di soluzione principale: il **valore di Shapley**.

Esempio (tre imprese). Tre imprese A, B, C possiedono ciascuna una parte della tecnologia per sviluppare un prodotto. Due imprese insieme guadagnano 100; una da sola guadagna 0; tutte e tre insieme guadagnano comunque 100. La **funzione caratteristica** v assegna a ogni coalizione il suo valore:

$$\begin{aligned} v(\{A\}) = v(\{B\}) = v(\{C\}) &= 0, \\ v(\{A,B\}) = v(\{A,C\}) = v(\{B,C\}) &= 100, \\ v(\{A,B,C\}) &= 100. \end{aligned}$$

- Se si forma la coalizione di tutti e tre e si divide in parti uguali: 33,33 a testa.
- Ma B e C da soli ottengono 100 e, dividendo, 50 a testa: stanno meglio abbandonando A. La divisione in parti uguali della grande coalizione non è stabile.

Esempio (variante: un giocatore più importante). Ora $v(\{A,B\}) = v(\{A,C\}) = 100$ ma $v(\{B,C\}) = 0$ (il resto invariato). Senza A non si produce nulla: A è più "importante" e la divisione in parti uguali non è più equa. Come dividere in modo equo? Con il **valore di Shapley**.

Approfondimento (calcolo del valore di Shapley nella variante). Il valore di Shapley (sarà definito formalmente più avanti) dà a ogni giocatore il suo *contributo marginale medio* su tutti gli ordini in cui i giocatori possono entrare nella coalizione. Con 3 giocatori gli ordini sono $3! = 6$:

Ordine	contributo di A	contributo di B	contributo di C
A, B, C	0	100	0
A, C, B	0	0	100
B, A, C	100	0	0
B, C, A	100	0	0
C, A, B	100	0	0
C, B, A	100	0	0
Media	66,67	16,67	16,67

> Esempio di lettura: nell'ordine B, C, A, A entra quando ci sono già B e C, e porta il valore da $v(\{B, C\}) = 0$ a $v(\{A, B, C\}) = 100$: contributo 100. Il totale è 100, cioè tutto il valore della grande coalizione. Nel caso simmetrico di prima si ottiene invece 33,33 a testa.

Applicazioni

- divisione equa (*fair division*);
- classifiche di importanza, potere di voto, . . . ;
- spiegabilità dell'IA: **SHAP** (valori di Shapley applicati alle feature di un modello);
- *network science*: massimizzazione dell'influenza, centralità, . . .

1.4.3 Scelta sociale computazionale

La **scelta sociale computazionale** (*Computational Social Choice*) si occupa del calcolo dei risultati delle regole di voto e dei problemi legati a rappresentare e raccogliere le preferenze dei giocatori. Domande tipiche:

- come si decide il vincitore di un'elezione?
- a quali condizioni la democrazia è possibile?

Esempio (paradosso di Condorcet). Tre votanti ordinano tre candidati:

Votante	1ª preferenza	2ª preferenza	3ª preferenza
Votante 1	A	B	C
Votante 2	B	C	A
Votante 3	C	A	B

Chi dovrebbe vincere? Confrontiamo i candidati a coppie, a maggioranza:

- A batte B (votanti 1 e 3);
- B batte C (votanti 1 e 2);
- C batte A (votanti 2 e 3).

La preferenza collettiva è **ciclica**: $A \succ B \succ C \succ A$. Anche se ogni votante ha preferenze coerenti (transitive), la maggioranza non le ha, e nessun candidato batte tutti gli altri.

Applicazioni

Elezioni, decisioni collettive, e nel machine learning i *random forest* e gli *ensemble* di modelli (ogni modello "vota" una predizione).

1.4.4 Mechanism design

Il **mechanism design** è una sorta di teoria dei giochi *al contrario*: invece di prendere un gioco come dato e analizzarne gli esiti, si vuole *progettare* un gioco che incentivi i giocatori a comportarsi nel modo desiderato.

> **Esempio (aste).** In un'asta si vuole che le persone dichiarino il *vero* prezzo a cui comprerebbero un oggetto o venderebbero un servizio.

Casi di successo: eBay, Google Ads, appalti pubblici e aste.

1.4.5 Matching markets

Nei **mercati di abbinamento** si vogliono abbinare agenti ad altri agenti, o agenti a oggetti, beni, servizi, garantendo qualche proprietà sociale desiderabile. Esempi: studenti e alloggi universitari, pazienti e donatori di rene, lavoratori e compiti. Nel corso si studierà un algoritmo premiato con il Nobel per lo **stable matching**.

1.5 Giochi combinatori

> **Definizione 1.5 (Gioco combinatorio).** Un **gioco combinatorio** ha le seguenti proprietà:
>
> 1. **due giocatori**, di solito chiamati Giocatore 1 e Giocatore 2, oppure MAX e MIN;
> 2. **informazione perfetta**: entrambi conoscono l'intero stato del gioco;
> 3. **nessun elemento casuale**: niente dadi, mazzi mescolati o eventi aleatori;
> 4. **finito**: il gioco termina sempre (niente partite infinite);
> 5. **turni alternati**: i giocatori muovono uno alla volta;
> 6. **somma zero**: il guadagno di un giocatore è esattamente la perdita dell'altro.

Esempi: scacchi, dama, go, tris (*tic-tac-toe*).

1.5.1 Dalla teoria dei giochi al giocare

Problema centrale: dato lo stato attuale del gioco, qual è la mossa *migliore*?

La mossa migliore è quella che porta al miglior esito possibile *assumendo che anche l'avversario giochi in modo perfetto*. L'idea chiave è modellare l'avversario come intelligente e capace di dare sempre la contromossa migliore.

> **Osservazione (perché assumere un avversario perfetto).** Se un giocatore vince contro la **risposta migliore** (*best response*) dell'avversario, vince anche contro qualunque risposta non ottimale. Ragionare sul caso peggiore dà quindi una *garanzia*: il risultato non può essere peggiore di quello calcolato.

1.5.2 L'albero di gioco

Definizione 1.6 (Albero di gioco). Ogni gioco di questo tipo si può rappresentare come un **albero di gioco**:

- i **nodi** sono stati del gioco (una configurazione della scacchiera);
- gli **archi** sono mosse, che portano da uno stato a un altro;
- la **radice** è lo stato attuale;
- le **foglie** sono stati terminali (vittoria di qualcuno o pareggio).

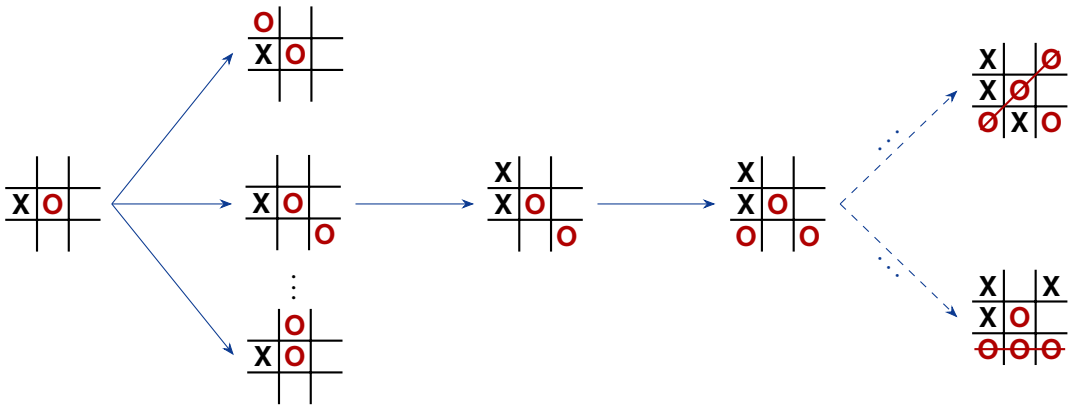


Figura 1.1: Una porzione dell'albero di gioco del tris. A destra due foglie in cui vince O.

1.5.3 Un'IA per il tris

Servono tre elementi:

1. un modo per **rappresentare** la scacchiera e lo stato del gioco;
2. una **funzione di valutazione** per sapere chi vince;
3. una **strategia di ricerca** nell'albero di gioco.

Funzione di punteggio. Il gioco è piccolo e a somma zero, quindi si assegna un punteggio agli stati finali:

$$\text{valore}(s) = \begin{cases} +1 & \text{se vince MAX,} \\ -1 & \text{se vince MIN,} \\ 0 & \text{in caso di pareggio.} \end{cases}$$

La grandezza dei numeri non conta: conta solo l'*ordine* ($+1 > 0 > -1$).

Rappresentazione. La scacchiera è una matrice 3×3 di numeri: X $\to 1$, O $\to 2$, cella vuota $\to 0$. Servono una funzione che controlla se c'è un tris su una riga, colonna o diagonale, e una che controlla se la scacchiera è piena.

1.5.4 L'algoritmo Minimax

Minimax è un algoritmo ricorsivo che esplora l'albero di gioco per determinare la mossa ottimale.

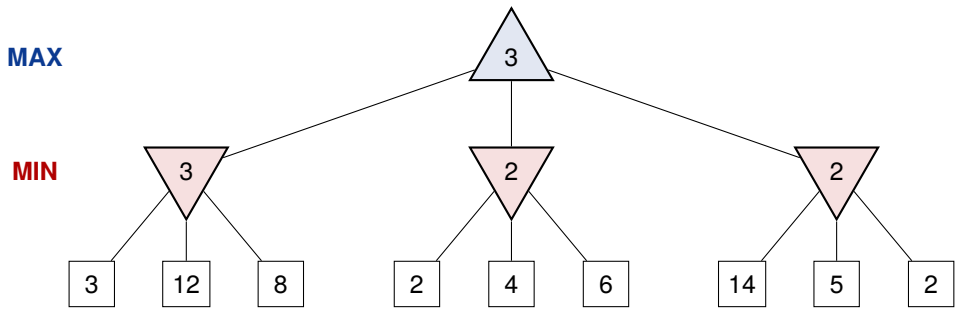
- Il giocatore **MAX** vuole *massimizzare* il punteggio: sceglie la mossa con il valore più alto restituito dalla chiamata ricorsiva.
- Il giocatore **MIN** vuole *minimizzare* il punteggio (che è quello di MAX): sceglie la mossa con il valore più basso.

```
def minimax(board, player):
    if board is leaf node:
        return value(board)
    if player = MAX:
        best = −∞
        for each move available:
            new_board = apply move to board
            val = minimax(new_board, MIN)
            best = max(val, best)
        return best
    else:
        best = +∞
        for each move available:
            new_board = apply move to board
            val = minimax(new_board, MAX)
            best = min(val, best)
        return best
```

In formula, il valore minimax di uno stato s è:

$$\mathrm{MM}(s) = \begin{cases} \text{valore}(s) & \text{se } s \text{ è terminale,} \\ \max_{s' \in \text{figli}(s)} \mathrm{MM}(s') & \text{se in } s \text{ muove MAX,} \\ \min_{s' \in \text{figli}(s)} \mathrm{MM}(s') & \text{se in } s \text{ muove MIN.} \end{cases}$$

Approfondimento (esempio numerico). Un albero a due livelli (MAX alla radice, MIN al livello sotto). Si parte dalle foglie e si risale: ogni nodo MIN prende il minimo dei figli, la radice MAX il massimo.



I tre nodi MIN valgono $\min(3,12,8) = 3$, $\min(2,4,6) = 2$, $\min(14,5,2) = 2$; la radice vale $\max(3,2,2) = 3$. MAX sceglie quindi la prima mossa.

1.5.5 I limiti di Minimax

Minimax nella versione base esplora *tutto* l'albero. Già per il tris l'albero completo ha 549 946 nodi (partendo dalla scacchiera vuota). Molti rami però sono inutili: se un ramo è già sicuramente peggiore di un'alternativa trovata prima, non serve esplorarlo.

1.5.6 Potatura Alpha-Beta

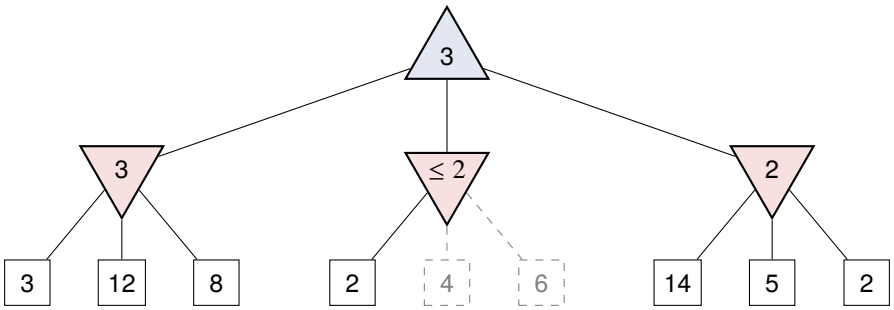
Si tengono traccia di due valori, i migliori trovati finora lungo il percorso:

- α: il valore migliore (più *alto*) che **MAX** può garantirsi al livello corrente o sopra;
- β: il valore migliore (più *basso*) che **MIN** può garantirsi al livello corrente o sopra.

All'inizio $\alpha = -\infty$ e $\beta = +\infty$. Appena $\alpha \geq \beta$, il nodo corrente non potrà mai essere raggiunto con gioco ottimale e i figli restanti si possono **potare** (`break`).

```
def minimax(board, player, alpha, beta):
    if board is leaf node:
        return value(board)
    if player = MAX:
        best = −∞
        for each move available:
            new_board = apply move to board
            val = minimax(new_board, MIN, alpha, beta)
            best = max(val, best)
            alpha = max(best, alpha)
            if alpha >= beta:
                break
        return best
    else:
        best = +∞
        for each move available:
            new_board = apply move to board
            val = minimax(new_board, MAX, alpha, beta)
            best = min(val, best)
            beta = min(best, beta)
            if alpha >= beta:
                break
        return best
```

> **Approfondimento (alpha-beta sull'esempio numerico).** Riprendiamo l'albero di prima, esplorando i figli da sinistra a destra.
>
> 1. Primo nodo MIN ($\alpha = -\infty, \beta = +\infty$): vede 3, 12, 8 e restituisce 3. La radice aggiorna $\alpha = 3$.
> 2. Secondo nodo MIN ($\alpha = 3$): la prima foglia vale 2, quindi $\beta = 2$. Ora $\alpha = 3 \geq \beta = 2$: **potatura**. Le foglie 4 e 6 non vengono visitate. *Perché è corretto*: questo nodo MIN varrà al massimo 2, ma MAX ha già un'alternativa che vale 3, quindi non sceglierà mai questo ramo.
> 3. Terzo nodo MIN ($\alpha = 3$): vede 14 ($\beta = 14$), 5 ($\beta = 5$), 2 ($\beta = 2$) e restituisce 2.
>
> Il risultato è 3, identico a Minimax, ma con due foglie in meno. Il risultato di alpha-beta è *sempre* identico a quello di Minimax: cambia solo quanto lavoro si fa.



Osservazione (l'ordine delle mosse conta). Quanto si pota dipende dall'ordine in cui si esplorano i figli. Se le mosse migliori vengono esplorate per prime, α e β si stringono subito e si pota molto di più. Nel caso ideale alpha-beta visita circa $b^{m/2}$ nodi invece di b^m (b = numero di mosse per stato, m = profondità): a parità di tempo si può cercare circa due volte più in profondità.

1.5.7 Oltre alpha-beta

Alpha-beta può ridurre l'albero di oltre il 99%, ma alcuni giochi restano troppo grandi (scacchi, go, . . .). Tecniche usate in pratica:

- **ricerca a profondità fissa** / *iterative deepening*: ci si ferma a una certa profondità, aumentandola finché c'è tempo;
- **valutazione euristica** degli stati: una stima del valore di stati non terminali;
- *quiescence search*: si continua a cercare oltre il limite solo nelle posizioni "instabili" (es. in mezzo a uno scambio di pezzi);
- *hashing* e *transposition tables*: si memorizza il valore di stati già visti, raggiungibili con sequenze di mosse diverse;
- *move ordering*: si provano prima le mosse promettenti, per potare di più;
- **basi di conoscenza**: aperture e tabelle dei finali negli scacchi.

1.6 Laboratorio: Minimax per il tris

LAB

[Lab_Minimax_blank.ipynb]

Obiettivo: implementare Minimax (e poi alpha-beta) per il tris, farlo giocare contro sé stesso e contro un umano. Il codice qui sotto è una soluzione completa ed è stata verificata: le asserzioni del notebook passano e le due IA pareggiano sempre.

1.6.1 Rappresentazione e condizioni di vittoria

La scacchiera è una lista di liste 3×3: `0` = vuota, `1` = X, `2` = O. `check_winner` restituisce `1` o `2` se quel giocatore ha un tris, altrimenti `0`. `is_full` dice se non ci sono celle vuote.

```python
def check_winner(board):
    lines = []
    lines.extend(board)                                  # righe
    lines.extend([[board[i][j] for i in range(3)]
                  for j in range(3)])                    # colonne
    lines.append([board[i][i] for i in range(3)])        # diagonale
    lines.append([board[i][2 - i] for i in range(3)])    # antidiagonale
    for line in lines:
        if line[0] != 0 and line[0] == line[1] == line[2]:
            return line[0]
    return 0

def is_full(board):
    return all(cell != 0 for row in board for cell in row)
```

> **Attenzione (la cella vuota).** Il controllo `line[0] != 0` è essenziale: senza, una riga vuota `[0,0,0]` verrebbe vista come "tre uguali" e la funzione restituirebbe un vincitore inesistente.

1.6.2 Minimax

X è MAX, O è MIN. La funzione `score` traduce il vincitore nel punteggio $+1/-1/0$. Invece di copiare la scacchiera a ogni mossa, si *modifica sul posto* e poi si **annulla la mossa** (*rollback*) prima di provare la successiva.

```python
from math import inf

def score(board):
    w = check_winner(board)
    if w == 1:
        return 1       # vince X (MAX)
    if w == 2:
        return -1      # vince O (MIN)
    return 0           # pareggio

def minimax(board, is_maximizing):
    if check_winner(board) or is_full(board):   # nodo foglia
```

```
        return score(board)
    if is_maximizing:
        best = -inf
        for i in range(3):
            for j in range(3):
                if board[i][j] == 0:
                    board[i][j] = 1                          # prova la mossa
                    best = max(best, minimax(board, False))
                    board[i][j] = 0                          # annulla la mossa
        return best
    else:
        best = inf
        for i in range(3):
            for j in range(3):
                if board[i][j] == 0:
                    board[i][j] = 2
                    best = min(best, minimax(board, True))
                    board[i][j] = 0
        return best
```

Attenzione (pseudocodice del notebook). Nel notebook, nel ramo MAX, il `return best` è indentato *dentro* il ciclo `for`: così la funzione restituirebbe il valore dopo aver provato solo la prima mossa. Il `return` va dopo il ciclo, come nelle slide e nel codice qui sopra.

1.6.3 Scegliere la mossa migliore

`minimax` restituisce un *valore*, non una mossa. `find_best_move` prova ogni mossa legale, valuta la posizione risultante con `minimax` e tiene quella con il valore più alto (se muove il giocatore 1) o più basso (se muove il giocatore 2).

```
def find_best_move(board, player):
    move = (-1, -1)                      # nessuna mossa se la scacchiera è piena
    best = -inf if player == 1 else inf
    for i in range(3):
        for j in range(3):
            if board[i][j] == 0:
                board[i][j] = player
                # dopo la mossa di 'player' tocca all'avversario:
                # se ha mosso X (1) ora minimizza O, e viceversa
                val = minimax(board, player == 2)
                board[i][j] = 0
                if (player == 1 and val > best) or (player == 2 and val < best):
                    best, move = val, (i, j)
    return move
```

Attenzione (errore tipico). Dopo aver piazzato la mossa di `player` bisogna chiamare `minimax` con il turno dell'*avversario*. Chiamarla con `is_maximizing = (player == 1)` è un errore comune: l'IA valuterebbe le posizioni come se toccasse di nuovo a lei.

1.6.4 Minimax contro sé stesso

Il notebook fornisce il ciclo di gioco: i giocatori si alternano chiamando `find_best_move` finché qualcuno vince o la scacchiera è piena. Con due giocatori ottimali la partita finisce **sempre in pareggio**. Con l'implementazione sopra la partita è:

$$(0,0),\ (1,1),\ (0,1),\ (0,2),\ (2,0),\ (1,0),\ (1,2),\ (2,1),\ (2,2) \quad \Rightarrow \quad \text{pareggio}$$

X	X	O
O	O	X
X	O	X

> **Approfondimento (perché il pareggio).** In ogni gioco combinatorio finito si può calcolare il valore di ogni stato risalendo dalle foglie (**induzione all'indietro**, *backward induction*), che è esattamente ciò che fa Minimax. Ne segue che uno dei due giocatori ha una strategia che garantisce la vittoria, oppure entrambi possono garantirsi almeno il pareggio (teorema di Zermelo). Per il tris vale la seconda: il valore minimax della scacchiera vuota è 0.

1.6.5 Minimax con potatura alpha-beta

Si aggiungono i parametri `alpha` e `beta` e il controllo di potatura dopo ogni figlio. Il resto del codice (incluso `find_best_move`) non cambia, salvo chiamare `minimax_ab` al posto di `minimax`.

```python
def minimax_ab(board, is_maximizing, alpha=-inf, beta=inf):
    if check_winner(board) or is_full(board):
        return score(board)
    if is_maximizing:
        best = -inf
        for i in range(3):
            for j in range(3):
                if board[i][j] == 0:
                    board[i][j] = 1
                    best = max(best, minimax_ab(board, False, alpha, beta))
                    board[i][j] = 0
                    alpha = max(alpha, best)
                    if alpha >= beta:
                        return best              # potatura
        return best
    else:
        best = inf
        for i in range(3):
            for j in range(3):
                if board[i][j] == 0:
                    board[i][j] = 2
                    best = min(best, minimax_ab(board, True, alpha, beta))
                    board[i][j] = 0
                    beta = min(beta, best)
                    if alpha >= beta:
                        return best              # potatura
        return best
```

Attenzione (`break` con due cicli annidati). Qui le mosse si scorrono con *due* cicli (`i` e `j`). Un `break` uscirebbe solo dal ciclo interno e l'esterno continuerebbe a esplorare le righe successive, annullando parte della potatura. Per questo si usa `return best` (va bene perché la mossa è già stata annullata). In alternativa si può prima costruire la lista delle celle vuote e usare un solo ciclo.

Risultati misurati. Stessa partita (pareggio), stesse mosse, ma molto meno lavoro:

	Minimax	Alpha-beta
Nodi visitati dalla scacchiera vuota	549 946	18 297
Nodi visitati in un'intera partita IA contro IA	618 175	36 684
Tempo della partita (Python)	circa 2,9 s	circa 0,17 s

Alpha-beta visita circa il 3% dei nodi di Minimax, cioè ne pota circa il 97%.

1.6.6 Giocare contro l'IA

L'ultima cella del notebook permette di sfidare l'algoritmo inserendo le mosse come `riga colonna` (indici da 0). Se l'implementazione è corretta **non si può mai vincere**: al massimo si pareggia.

Riepilogo della lezione

- Teoria dei giochi = decisioni tra più agenti razionali; AGT = progettare algoritmi in ambienti strategici.
- Aree del corso: giochi non cooperativi (equilibrio di Nash), coalizionali (valore di Shapley), scelta sociale (paradosso di Condorcet), mechanism design, matching.
- Dilemma del prigioniero: efficienza (Pareto) e stabilità (Nash) possono non coincidere.
- Giochi combinatori: 2 giocatori, informazione perfetta, niente caso, finiti, a turni, a somma zero. Si rappresentano con l'albero di gioco.
- Minimax: valore di uno stato = max (MAX) o min (MIN) dei valori dei figli.
- Alpha-beta: stesso risultato, pota i rami con $\alpha \geq \beta$. Nel tris visita 18 297 nodi invece di 549 946.

Prossima lezione: teoria dell'utilità e decisioni in condizioni di incertezza; giochi strategici e formalizzazione della *best response*.